dPen Maths

Table of Contents

1. What Is RedPen Maths?
2. The Big Picture — System Overview
3. Tech Stack at a Glance
4. The Demo Marking Flow (What Users See Today)
    - 4.1 Student Draws on Canvas
    - 4.2 Canvas Captured as Image
    - 4.3 Image Sent to the AI
    - 4.4 AI Returns Structured Marks
    - 4.5 Answer Protection (3 Layers)
    - 4.6 Red Ink Animated on Screen
    - 4.7 Tutor Speaks the Feedback
    - 4.8 Lesson Mode (Teach Me This Step)
5. The Engine Marking Pipeline (For Practice Exams)
    - 5.1 Stage 1: Reader — Handwriting to LaTeX
    - 5.2 Stage 2: Matcher — Aligning to the Rubric
    - 5.3 Stage 3: Checker — SymPy Math Verification
    - 5.4 Stage 4: Judge — Qualitative Checks
    - 5.5 Stage 5: Scorer — Computing the Grade
6. Abuse Prevention & Cost Control
    - 6.1 Cloudflare Turnstile (Bot Gate)
    - 6.2 Session Cookies
    - 6.3 Redis Rate Limiting (3 Counters)
    - 6.4 Cost True-Up After Each AI Call
7. The Landing Page & User Journey
8. Waitlist, Payments & Database
9. Text-to-Speech System
10. Dual AI Provider Architecture
11. Infrastructure & Deployment
12. What's Live vs. Under Construction

1. What Is RedPen Maths?

RedPen Maths is an **AI-powered calculus tutor** that grades a student's handwritten working **line by line**, finds the **first wrong step**, marks it in **red ink** (just like a real teacher would), and **explains the mistake out loud**.

Think of it as having a patient tutor sitting next to you, looking over your shoulder with a red pen. You write your working. The tutor watches. When you make a mistake, they circle it, cross out the wrong part, write the correction, and say "Here's where you went off track — let me show you."

Who Is It For?

- US college students taking **Calculus I**
- Topics: derivatives, integrals, limits, related rates, optimization, the Fundamental Theorem of Calculus

Business Model

- **Free tier:** first marked practice exam is free
- **Semester pass:** $49 one-time
- **Monthly:** $14.99/month

What Exists Today

- **A landing page** with marketing copy, FAQ, waitlist signup
- **A live interactive demo** with 3 sample questions (derivative, limit, integral)
- **Practice exams** are under construction (engine built, not yet exposed)

2. The Big Picture — System Overview

At its core, RedPen Maths takes a **picture of handwritten maths**, sends it to an **AI model** along with the question and answer key, and gets back **structured feedback** (what to circle, what to strike out, what to write, what to say). Then it **animates** that feedback as hand-drawn red ink on the student's page and **speaks** the explanation.

The system has **two separate marking pipelines**:

Aspect	Demo Marking	Engine Marking
Purpose	Live interactive demo on the landing page	Full practice exam grading (under construction)
How it works	One AI vision call: image in, annotations out	4-stage pipeline: Reader, Matcher, Checker (SymPy), Judge, Scorer
Math verification	AI judgment only (no formal proof)	SymPy computer algebra (provably correct)
Speed	2-5 seconds	30-60 seconds
Cost	~1 penny per check	Higher (multiple AI calls + SymPy)
Status	**Live**	**Built, not exposed**

Demo Marking vs. Engine Marking — two systems for two different needs

3. Tech Stack at a Glance

Layer	Technology	What It Does Here
Framework	Next.js 16 (App Router)	The web application framework — serves pages and API routes
Language	TypeScript + React 19	Type-safe code with modern React for the UI
AI (Primary)	Anthropic (Claude Sonnet 5)	Reads handwriting, marks mistakes, teaches steps
AI (Alternate)	OpenAI (GPT-6-Luna)	Same role — switchable via one environment variable
Speech	OpenAI TTS (gpt-4o-mini-tts)	Converts tutor's text feedback into spoken audio
Math Engine	Python / SymPy (FastAPI)	Symbolic algebra: proves if two math expressions are equal
Database	PostgreSQL	Stores waitlist signups, orders, daily demo stats
Cache / Limits	Redis	Atomic counters for rate limiting and cost budgets
Bot Protection	Cloudflare Turnstile	Invisible CAPTCHA — proves visitor is human
Payments	Paddle	Checkout, subscriptions, refunds (currently off)
Email	Resend	Waitlist confirmations, purchase receipts
Validation	Zod 4	Validates all data shapes (AI output, API input, config)
Hosting	Railway	Deploys the Next.js app + runs database migrations

4. The Demo Marking Flow

What Users See Today

This is the **heart of the product** right now — the interactive demo on the landing page. Here's exactly what happens from the moment a student starts drawing to when they hear the tutor's voice:

4.1 Student Draws on Canvas

The demo presents a **900 x 620 pixel canvas** (like a sheet of paper) with a question written at the top. The student picks one of three demo questions:

1. Derivative: f(x) = x^3 + x^2. Find f'(2). [Answer: 16]
2. Limit: Find lim (x^2-9)/(x-3) as x approaches 3. [Answer: 6]
3. Integral: Evaluate the integral of x^2 dx from 0 to 1. [Answer: 1/3]

The student writes with their finger (on mobile) or mouse (on desktop). Each stroke is recorded as a series of points and stored in a **Page model** — an ordered list of items (strokes, tutor marks, images) that can be undone, replayed, and rendered.

4.2 Canvas Captured as Image

After the student **pauses writing for 2.5 seconds**, the system automatically decides it's time to check their work. (Or the student can press the Check button to force it.)

The entire canvas is rendered as a **PNG image** (base64 data URL, max 1.5MB). Coordinate rulers are added around the edges so the AI knows the exact pixel positions. The question band at the top is excluded from the capture — the AI already knows the question.

4.3 Image Sent to the AI

The PNG is sent to **POST /api/mark** along with:

- **questionId** — which of the 3 demo questions the student picked
- **space** — the canvas dimensions (width and height)
- **layout** — where free space is on the page (so the tutor knows where to write)
- **prior** — up to 40 previous tutor annotations (so the AI remembers what it already drew)
- **transcript** — last 8 lines of spoken dialogue (so the conversation flows naturally)
- **lesson / force flags** — whether this is a "teach me" request or a forced check

On the server, the handler:

1. Validates the session cookie (are you a verified human?)

2. Looks up the question (is this a real demo question?)
3. Validates the image (is it actually a PNG under 1.5MB?)
4. Checks rate limits (have you used too many checks today?)
5. Charges an up-front cost estimate (1p for a check, 3p for a lesson)
6. Builds a detailed instruction (combining the question, answer key, layout, and rules)
7. Calls the AI model with the image + instruction

What the AI Is Actually Told

The AI receives a massive, carefully crafted **system prompt** that defines its entire personality and behavior. Here's what's in it:

- **Persona:** "You are a tutor holding a red pen, looking over a student's shoulder."
- **Handwriting rules:** Detailed heuristics for reading student handwriting (e.g., "a 2 can look like a Z", "a badly drawn integral sign might be an S")
- **Assessment rules:** When to respond and when to stay silent. The tutor only speaks when the student has actually written something worth checking.
- **Marking rules:** Find the **first** wrong step only. Strike through the wrong part. Write the correction nearby. Never mark more than one mistake at a time.
- **Teaching rules:** How to structure a lesson (3-7 steps). Use the student's own drawing to teach. Draw diagrams when helpful.
- **Answer key:** The correct answer, every acceptable form of it, and the mark scheme.
- **The golden rule:** **NEVER reveal the final answer.** Not in speech, not in writing, not even by implication. The banned words include "solve", "answer", "answers", and every numerical form of the correct value.

4.4 AI Returns Structured Marks

The AI doesn't return free text — it returns **structured JSON** that the code can work with. The response is validated by Zod schemas. Here's what comes back for a normal check:

- **worthResponding** (true/false) — is there something to respond to?
- **correct** (true/false) — is the student's work correct so far?
- **speech** (string) — what the tutor says out loud
- **annotations** (array) — the red ink marks to draw. Each is one of 9 types:
    - **strike** — cross out wrong work
    - **write** — write text at coordinates (the correction)
    - **circle** — circle something to draw attention
    - **arrow** — draw an arrow between two points
    - **text, line, rect, path, ring** — other drawing primitives

For a lesson ("teach me this step"), the response is an array of **steps**, each with a **say** (what to speak) and **write** (what to draw) for that step.

4.5 Answer Protection — Defense in Depth

The most critical rule is: **the tutor must never reveal the final answer**. This is enforced at **three separate layers**:

1

Prompt Layer (AI Instructions)

The system prompt explicitly bans saying or writing the answer, any spoken form of it (e.g., "sixteen", "one third"), and words like "solve" and "answer".

2

Schema/Code Layer (Post-Processing)

After the AI responds, server code scans every annotation and every sentence. It uses a 'canonical' normalizer that catches number words ("five twelfths" becomes "5/12"), fractions, mixed numbers, and various formats. Any annotation or sentence that reveals the answer is silently dropped.

3

Handler Layer (Final Fallback)

If the speech was replaced (because it revealed the answer), safe fallback messages are used instead: "That's right!" for correct work, "Take another look at the part I've marked" for wrong work, or "See if you can finish it yourself" for lessons.

4.6 Red Ink Animated on Screen

Once the validated annotations come back, they don't just appear — they're **animated** as if a hand is drawing them in real time. This is handled by the ink rendering system.

The animation system works like this:

- **Strokes** are drawn point-by-point at 300 pixels/second, with slight speed wobbles (0.85x to 1.15x) for realism
- **Text** is revealed character-by-character at 95ms per character
- **Pen lifts** (pauses between marks) last 70-200ms
- **Humanization:** all paths are resampled with ease-in-out timing and low-frequency drift added — so circles aren't perfect, lines aren't straight, just like a real hand
- **Deterministic randomness:** the "wobble" is seeded from coordinates, so the same mark always wobbles the same way (consistent across renders)

There's also a **layout resolution** step before drawing: long arrows (>160px) are dropped, text that would overlap student work is repositioned to free space, and "ring" annotations ("circle what I just wrote") are resolved to actual coordinates.

4.7 Tutor Speaks the Feedback

While the red ink is being drawn, the tutor's voice speaks the feedback. The speech system has **three tiers**:

1. Neural voice (preferred): The text is sent to /api/speak, which calls OpenAI's text-to-speech API (gpt-4o-mini-tts model, "sage" voice). The tutor is described as a "patient British maths tutor". Audio streams back as MP3.

2. Browser synthesis (fallback): If neural voice isn't available (no API key, rate-limited, or error), the browser's built-in speech synthesis is used with a ranked list of preferred English voices.

3. Muted mode: If the user has muted, the system still waits a "reading time" delay (600ms + 260ms per word, max 3 seconds) so lesson pacing feels natural.

For lessons, the system **prefetches** the next step's audio while the current step is playing, so there's no gap between steps.

4.8 Lesson Mode ("Teach Me This Step")

When the student taps "Teach me this step", a different flow is triggered:

- The AI returns 3-7 **lesson steps** instead of a single response
- Each step has a **say** (spoken explanation) and **write** (annotations to draw)
- Steps play sequentially: speak step 1 while drawing step 1's marks, then speak step 2 while drawing step 2's marks, etc.
- The AI is told to teach using the student's own drawing — point to their work, draw diagrams on their page, build on what they've already done

- The lesson costs 3x a normal check (3p vs. 1p) because it uses more tokens

5. The Engine Marking Pipeline

For Practice Exams (Under Construction)

The demo marking is great for quick feedback, but it relies on AI *judgment* alone — the AI might say 2+2=5 is wrong, but it doesn't *prove* it mathematically. For actual exam grading, you need **provably correct math verification**. That's what the engine does.

The engine uses a **5-stage pipeline** where AI handles what it's good at (reading handwriting, understanding context) and a **computer algebra system (SymPy)** handles what it's good at (proving math is right or wrong).

Stage 1: Reader — Handwriting to LaTeX

1

Reader

Takes 1-4 photos of handwritten work. An AI model transcribes each line into LaTeX (the standard math notation language). For each line, it provides: the LaTeX text, a bounding box (where on the page it is), and a confidence level (high or low). Max 40 lines, 400 characters per line. If the handwriting is unreadable, it says "retake" instead.

Stage 2: Matcher — Aligning Lines to Rubric Points

2

Matcher

An AI model looks at the student's transcribed lines and the rubric (mark scheme) and figures out: (a) what mathematical operation each line represents (differentiate, integrate, substitute, evaluate, rewrite, take limit), and (b) which rubric point each line is attempting to earn. This alignment is critical for scoring.

Stage 3: Checker — SymPy Math Verification

This is the **mathematical brain** of the engine — a separate Python service running SymPy (a computer algebra system). It does NOT use AI — it uses **symbolic mathematics** to prove things.

It checks three things:

A) Transition Verification (Does each line follow from the previous one?)

For each consecutive pair of lines, the checker verifies the mathematical step. For example:

- **Rewrite:** Are the two expressions algebraically equal?

- **Differentiate:** Is line 2 the derivative of line 1?
- **Integrate:** Is line 2 the integral of line 1?
- **Substitute:** Does plugging in the value give the next line?
- **Take limit:** Does evaluating the limit give the next line?

Smart fallback: if the AI matcher labeled a step as "differentiate" but SymPy can't confirm it, the checker tries ALL other transition types. If any one proves correct, the step passes. This handles cases where the AI mis-labeled the step.

B) Equivalence Proving (Are two expressions the same value?)

The core math engine. Given two expressions, it tries three strategies:

1. Symbolic simplification: simplify(a - b) and check if it equals 0

2. For constants: evaluate both numerically and compare

3. For expressions with variables: test at 8 random points (from 60 attempts). If all 8 agree, it's RIGHT. If 2+ disagree, it's WRONG.

C) Rubric Point Verification

For each rubric point that has a CAS-checkable expected value ("equivalent", "equation", "antiderivative", "exact_value"), the checker verifies whether the student's line on that rubric point matches the expected answer.

Stage 4: Judge — Qualitative Checks

4 **Judge**

Some rubric points can't be checked by math alone — they require the student to STATE a reason (e.g., "by the chain rule" or "since the denominator approaches 0"). An AI model reads the student's lines and checks whether each required justification was stated. For each criterion, it reports: met or not met, which line, and a verbatim quote.

The judge runs **in parallel** with the SymPy checker (Stage 3) since they don't depend on each other. If the judge fails (AI error), the system degrades gracefully — those points become "unchecked" rather than failing the whole pipeline.

Stage 5: Scorer — Computing the Grade

5 **Scorer**

Combines all results from the checker and judge into final scores. Each rubric point gets a status: earned, earned (with follow-through), not earned, or unchecked.

Key scoring features:

- **Follow-through marking:** If a student makes one mistake early but then does all subsequent steps correctly with their wrong value, they can still earn marks for those subsequent steps. This is standard exam practice — you shouldn't lose ALL marks because of one arithmetic slip.
- **Dependency chains:** Some rubric points require others to be earned first. For example, you can't earn the "correct final answer" point unless you earned the "correct method" point.
- **Valid alternative method:** If the student used a different method than expected but all their lines form a proven chain from question to correct final answer (at least 3 math lines), ALL CAS points are awarded. You shouldn't lose marks for being creative.
- **Exam-wide penalties:** Missing +C in integration is penalized once for the whole exam, not per-problem. The scorer aggregates across all problems.

6. Abuse Prevention & Cost Control

Every AI call costs money. A single demo check costs about 1 penny, but if someone writes a bot that hammers the API 10,000 times, that's 100 GBP gone. The abuse prevention system has **four layers**:

6.1 Cloudflare Turnstile (The Bot Gate)

Before a visitor can use anything, they must pass a **Cloudflare Turnstile challenge** — an invisible CAPTCHA that proves they're human. This happens once per visit.

How it works in the UI:

1. When the page loads, the entire landing page is rendered but hidden behind an opaque overlay (content is still there for SEO crawlers)
2. The system first checks if there's already a valid session cookie
3. If not, it shows the Turnstile widget
4. On success, the Turnstile token is sent to the server
5. The server verifies the token with Cloudflare's API
6. If valid, a 24-hour session cookie is issued
7. The overlay disappears and the student can use the demo

6.2 Session Cookies (Proving You Passed the Gate)

After passing Turnstile, the visitor gets an **HMAC-signed session cookie** that lasts 24 hours. Every API call checks this cookie. The cookie contains an expiry timestamp and an HMAC signature — it can't be forged or extended. The comparison uses **timing-safe equality** to prevent timing attacks.

6.3 Redis Rate Limiting (Three Counters)

Even verified humans are limited. Three atomic Redis counters enforce daily caps:

Counter	Scope	Default Limit	What It Prevents
Check rate	Per visitor per day	10 checks	One person using too many checks
Visitor budget	Per visitor per day	40 pence	One person spending too much
Global spend	All visitors per day	1000p (10 GBP)	Total daily cost runaway

Key design: all three counters are charged **atomically** (all-or-nothing). If ANY limit is exceeded, ALL charges are **refunded** immediately. Visitors are identified by a **salted SHA-256 hash of their IP address** (with IPv6 /64 grouping for privacy). All counter keys auto-expire after 2 days.

6.4 Cost True-Up After Each AI Call

Before calling the AI, the system charges an **estimated cost** (1p for a check, 3p for a lesson). After the call completes, it calculates the **actual cost** from the token usage and charges the difference. This "true-up" ensures the budget accurately reflects real spending, even if some calls use more tokens than expected.

7. The Landing Page & User Journey

A visitor's journey through the site:

1 **Arrive & Prove Human**

Visitor lands on the page. Turnstile runs in the background. Once passed, the overlay lifts and they see the full landing page.

2 **See the Hero Animation**

At the top of the page, an animated demo loops continuously — showing student handwriting being drawn, followed by red-pen marking appearing. This plays with audio (muted by default) from a pre-recorded MP3. It respects 'prefers-reduced-motion' accessibility settings.

3 **Try the Interactive Demo**

The visitor scrolls to the demo section, picks a question, and writes their working. The full marking flow (Section 4) kicks in. If they hit the rate limit, the interactive canvas gracefully falls back to showing the hero animation instead.

4 **See Scoring & How It Works**

Below the demo: a scoring example table showing how marks are awarded, a 3-step "how it works" section (write your working, get red-pen feedback, learn from mistakes), and trust points.

5 **Pricing & Waitlist**

Pricing section (free first exam, $49 semester, $14.99/month), FAQ accordion, and a waitlist signup form. The form collects: email, course, school, exam date. A double opt-in confirmation email is sent via Resend.

The page is styled as a **sheet of exam paper** — cream background (#FFFDF7), handwriting-style accents (Caveat font), and a clean academic feel (Bricolage Grotesque for body text).

8. Waitlist, Payments & Database

Waitlist System

The waitlist is the primary conversion mechanism right now. Here's the flow:

1. Student fills the form (email, course, school, exam date)
2. POST /api/waitlist stores the record in PostgreSQL
3. A confirmation email is sent via Resend with a unique token
4. Student clicks the confirm link -> /api/waitlist/confirm
5. The record is marked as confirmed
6. An unsubscribe link is always included for compliance

Payment System (Paddle — Currently Off)

Payments are fully built but **feature-flagged off** (NEXT_PUBLIC_PAYMENTS_ENABLED). When enabled:

- A Buy button loads the **Paddle checkout overlay** (no redirect needed)
- Paddle sends **webhooks** for transaction.completed and refund events
- The server verifies Paddle's webhook signature and records orders
- Purchase receipt emails are sent via Resend
- There's a **pass cap** (200 max active orders) for early-bird scarcity

Database (PostgreSQL)

Three tables:

- **waitlist** — email, role, course, school, exam date, confirmation status, unsubscribe token
- **orders** — Paddle transaction ID, email, amount, currency, status (completed/refunded)
- **demo_stats** — daily counters: total checks, errors caught, teach taps

Important: No student work is stored in the database. The canvas images are sent to the AI and discarded. The database is purely for business operations.

9. Text-to-Speech System

The tutor's voice is a key part of the experience. Here's how it works end-to-end:

1. Client requests speech: After receiving a tutor reply, the TryIt component sends the speech text to POST /api/speak

2. Server validates: Checks session, sanitizes text (max 600 chars, strips control characters), rate-limits (1p per speech call, counts toward visitor budget but NOT toward the check rate limit)

3. OpenAI TTS call: The text is sent to OpenAI's /v1/audio/speech endpoint with: model gpt-4o-mini-tts, voice "sage", instructions "patient British maths tutor", MP3 format

4. Audio streams back: The MP3 audio is streamed directly to the client (no buffering the whole file first)

5. Client plays: An Audio element plays the stream. If it fails, falls back to browser synthesis

A GET to /api/speak returns whether voice is available (does the server have an OpenAI key?), so the client knows whether to offer the neural voice option.

10. Dual AI Provider Architecture

The entire marking system can run on **either Anthropic (Claude) or OpenAI (GPT)**, switchable via a single environment variable: **MARK_PROVIDER**.

Why two providers?

- **Resilience:** if one provider has an outage, switch to the other
- **Cost optimization:** compare pricing between providers
- **Quality comparison:** A/B test which model marks better

Aspect	Anthropic	OpenAI
Model	claude-sonnet-5	gpt-6-luna
SDK	@anthropic-ai/sdk (official)	Raw fetch (no SDK)
Structured output	Zod-based (zodOutputFormat)	JSON Schema (oneOf -> anyOf conversion)
Reasoning effort	'low' (check), 'medium' (lesson)	'low' (check), 'medium' (lesson)
Timeout	20s (check), 45s (lesson)	20s (check), 45s (lesson)
Cost per MTok (GBP)	3 / 15 (in / out)	0.1 / 0.4 (in / out)

Both providers share the exact same handler, prompt, schema validation, and post-processing. Only the actual API call differs. The provider selection happens once at module load time — it's not per-request.

11. Infrastructure & Deployment

Project Structure

Here's where everything lives:

Folder	Purpose
`app/`	Next.js pages and API routes (the web layer)
`app/api/`	Backend endpoints: /mark, /speak, /session, /waitlist, /paddle
`components/`	React components: TryIt, HeroDemo, HumanGate, WaitlistForm, etc.
`lib/`	Core business logic (the real brain of the app)
`lib/mark/`	Demo marking pipeline: handler, prompt, schema, providers
`lib/engine/`	Engine marking pipeline: reader, matcher, checker, judge, scorer
`lib/guard/`	Abuse prevention: turnstile, session, counter, limits
`content/`	Static content: demo questions, marketing copy, hero script
`services/checker/`	Python SymPy microservice (separate deployment)
`db/`	PostgreSQL schema
`scripts/`	Migration runner, evaluation scripts, E2E tests
`public/`	Static assets: hero audio, logos, icons
`docs/`	Design docs and decision records

Deployment

- **Main app:** Deployed on **Railway**. Pre-deploy hook runs database migrations.
- **SymPy checker:** Separate container (Dockerfile in services/checker/). Communicates with the main app via HTTP, authenticated by a shared HMAC secret.
- **Redis:** Managed instance on Railway (or wherever REDIS_URL points).
- **PostgreSQL:** Managed instance on Railway (or wherever DATABASE_URL points).

Key Environment Variables

The system is configured entirely through environment variables. Key ones:

- **MARK_PROVIDER** — 'anthropic' or 'openai' (which AI to use)
- **ANTHROPIC_API_KEY / OPENAI_API_KEY** — API credentials
- **DATABASE_URL / REDIS_URL** — infrastructure connections

- **TURNSTILE_SECRET_KEY** — Cloudflare bot protection
- **SESSION_SECRET** — HMAC key for session cookies
- **CHECKER_URL / CHECKER_SECRET** — SymPy service connection
- **DEMO_DAILY_CAP_GBP** — maximum daily AI spend (default 10 GBP)
- **NEXT_PUBLIC_PAYMENTS_ENABLED** — feature flag for Paddle payments

12. What's Live vs. Under Construction

Feature	Status	Notes
Landing page	**Live**	Exam-paper themed, fully responsive
Interactive demo (3 questions)	**Live**	Derivative, limit, integral
AI marking (demo mode)	**Live**	Anthropic or OpenAI, with answer protection
Red ink animation	**Live**	Hand-drawn feel, 9 annotation types
Text-to-speech (neural)	**Live**	OpenAI TTS with browser fallback
Lesson mode	**Live**	Multi-step teaching with synchronized speech + drawing
Bot protection + rate limiting	**Live**	Turnstile + session + 3 Redis counters
Waitlist signups	**Live**	With double opt-in email confirmation
Hero demo animation	**Live**	Looping animation + pre-recorded audio
Engine marking pipeline	**Built**	5-stage pipeline complete, not user-facing yet
SymPy checker service	**Built**	Docker container, tested, used by engine
Practice exams	**Not yet**	Question bank exists, UI not built
Payments (Paddle)	**Off**	Fully built, feature-flagged off

This document describes the system architecture and flow of RedPen Maths as of October 2026. It is a conceptual guide — not a code reference. For implementation details, read the source.